ARK SelectionTask: Perception Team

Session 2025-2026

—----------

1 General Instructions

The following tasks are intended to check your skills in the fields of Computer Vision and Path Planning. Feel free to ask any doubts in the WhatsApp group . The final solution must be submitted in GitHub with documentation of each task. Please use the provided document Template in overleaf to create documentation/report (template.zip). We had fun creating these tasks, and we hope that you'll enjoy them too. Do not copy code from friends, as we will use automated plagiarism checkers. Also, if you use any library functions, you might be asked to explain in detail how they work. Ping us for more problems if you don't find this challenging enough.

2 Tasks

2.1 Vision-Based Line Follower

A. Environment Setup & Prerequisites

To successfully execute this task, you must ensure your development environment is correctly configured:

- **Software Version:** You must use **MATLAB R2024b**.
- **Required Dependencies:** Install the **Simulink Support Package for Parrot Minidrones and more related dependencies** via the MATLAB Add-On Explorer.

B. Project Initialization

You will be provided with the full competition project folder in the standard MATLAB Project format. Follow these strict steps to open the working environment:

1. Open MATLAB.
2. Locate and double-click the `.prj` file to initialize the project environment.
3. In the project file browser, navigate to the `mainModel` folder.
4. Open the `parrotMinidroneCompetition.slx` file.

C. Objective

Your goal is to implement a **Line Follower** algorithm. The drone must process the downward-facing camera feed to identify a track line on the ground and navigate along it autonomously.

Implementation Constraints:

- You simply **have** to modify the **Image Processing System** within the Simulink model.
- The output of your block must be the **Line Angle** (deviation from the center), which the flight controller will use to steer the drone.

D. Grading Criteria

Your submission will be evaluated automatically based on the telemetry data sent from your image processing block:

- **Primary Metric:** The accuracy of the **Angle** sent to the controller.
- **Fail Condition:** If the drone loses the line or crashes due to incorrect angle calculation, the run is terminated.

E. Simulation & Testing

You can change the track as well:

- **Changing Tracks:** Go to the **"Project Shortcuts"** tab in the MATLAB toolstrip to change the map line or open the Track Builder to test different curve severities.

F. Bonus Challenge: The Landing Zone

- **Objective:** Detect the **Circle** marker at the end of the line.
- **Requirement:** Upon detecting the circle, your system must trigger a "Landing" flag or specific stop command (depending on the interface provided in the template).
- **Points:** Extra credit will be awarded for a successful stop/land maneuver exactly over the circle.

Download the .prj file from here → Perception task

2.2 Noise Filtering

A. Objective

You are provided with a 2 corrupted image file (`iron_man_noisy.jpg`) (scenary_noise.jpg). The First image contains a line drawing of a subject (Iron Man), but the signal is heavily degraded by high-frequency "salt noise" (random white dots scattered across the frame). The

Your goal is to write a script that filters out these artifacts to restore the clean line drawing as much as possible.

B. The Challenge: This is a balancing act.

- **The Sweet Spot:** You must find the method and parameters that maximize noise removal while minimizing damage to the original figure.

C. Instructions

1. **Input:** Load the provided noisy image.
2. **Processing:** Use **OpenCV** (Python) to implement a noise reduction algorithm.
    - *Hint:* Standard Gaussian blurring might blur the edges too much. Consider **Morphological Operations** or **Non-Linear Filters** (like Median Blur).
3. **Output:** Save the processed image

D. Deliverables

- The source code (Python script).
- The final output image.

E. Grading Criteria

You will not be graded on achieving a "perfect" image. We are looking for:

- **Signal-to-Noise Ratio**

- **Feature Preservation**
- **Justification**

2.3 Medial Axis Detection of Moving Objects

A. Objective

Your primary task is to process video clips of a moving surgical tool to highlight its Medial Axis (the central skeletal line) on the original frames.

• You can access the videos [here].

• We'll be working on the frames extracted from the video. Please refer to the following [link].

B. Required Pipeline:

The document specifies **four distinct** stages for this project:

1. **Background Subtraction:** You must separate the moving foreground object (the tool) from the static background in the video sequence. Please refer to the following link for [background subtraction].
2. **Image Cleaning:** The subtracted image will contain noise and imperfections. You are required to use morphological image processing techniques (such as erosion and dilation) to clean the image and obtain a clear silhouette of the moving object. Please refer to the following link for [morphological image processing techniques].
3. **Edge & Line Detection:**
    - First, identify edges in the cleaned image using derivatives.
    - Second, identify the straight edges of the tool using the Hough Line Transform.

    Please refer to the following links for [derivatives], [hough line transformation].

4. **Medial Axis Localization:** Using the detected straight edges, compute the medial axis of the object and overlay it visually onto the original video frame.

C. Critical Constraints

- **No Built-in Hough Transform:** You are explicitly forbidden from using OpenCV's built-in Hough Line Transform implementation. You must code this algorithm from scratch to demonstrate your understanding of the concept.

- **Allowed Tools:** You may use OpenCV for basic operations, but the core algorithmic logic for line detection must be your own.

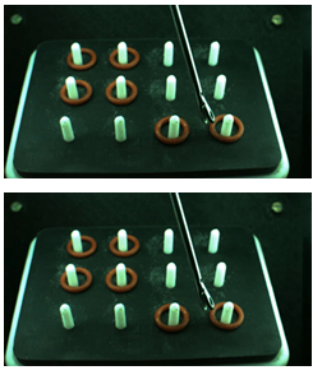
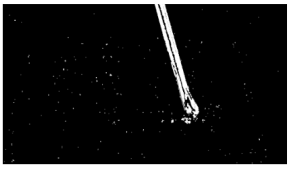


Figure 1: Two consecutive frames showing slight movement in the tool.



(a) Figure (1) frames after background subtraction.

(b) Figure (a) after cleaning using Opening operation. (You can try any other method of your choice.)

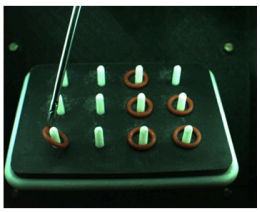


Figure 3: Initial frame

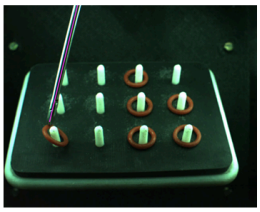


Figure 4: Medial axis marked on the initial frame

Bonus Task: The "Full Stack" Simulation Setup

A. Objective:

In the world of autonomous aerial robotics, testing code on real hardware is the last step, not the first. Your goal is to configure the industry-standard "Software In The Loop" (SITL) simulation stack on your machine.

B. The Stack (Requirements)

You must successfully install and interface the following four components on **Ubuntu 22.04**:

1. **ROS 2 (Distribution: Humble Hawksbill):**
    - *What it is:* The middleware that acts as the "nervous system" of the robot, allowing different software nodes (vision, control, navigation) to talk to each other.
    - *Why we use it:* It is the global standard for robotics research and industry, offering real-time capabilities and a massive ecosystem of libraries.
    - *Documentation:* ROS 2 Humble Installation Guide
2. **Gazebo (Version: Harmonic):**
    - *What it is:* A high-fidelity physics simulator that calculates gravity, aerodynamics, and collisions.
    - *Why we use it:* It allows us to crash a virtual drone 1,000 times without costing a single rupee. Harmonic is the latest generation of the Gazebo simulator, offering improved rendering and physics sensors.
    - *Documentation:* Gazebo Harmonic Docs
3. **PX4 Autopilot (SITL Mode):**
    - *What it is:* The actual flight control software that runs on real flight controllers (like the Pixhawk). In SITL mode, it runs as a program on your laptop.
    - *Why we use it:* It provides professional-grade flight stabilization and mission modes. If your code works with PX4 SITL, it will work on the real drone with minimal changes.
    - *Documentation:* PX4 ROS 2 User Guide
4. **QGroundController (QGC):**
    - *What it is:* The "Cockpit" dashboard.
    - *Why we use it:* To visualize the drone's position on a map, upload waypoints, and monitor battery/sensor health in real-time.
    - *Documentation:* QGC Installation Guide

C. The Challenge:

The "Bridge" : Installing these individually is easy. The challenge is making them talk to each other.

- You must set up the **MicroXRCE-DDS Agent** to bridge PX4 topics to ROS 2.
- You must ensure the `ros_gz_bridge` is correctly configured so ROS 2 can spawn models in Gazebo Harmonic.

D. Deliverable Submit a single screenshot showing the "Quad-View":

1. **Terminal 1:** Running the PX4 SITL instance.
2. **Gazebo Harmonic:** Showing a 3D drone hovering.
3. **QGroundController:** Showing the drone as "Connected" and "Armed."
4. **Terminal 2:** Running `ros2 topic list` showing the drone's sensor data (e.g., `/fmu/out/vehicle_odometry`) being published.

For any queries contact:

Parth Bhardwaj- 91230 08287

Aryan Kumar- 81263 95547

Also join the Whatsapp group for any doubts - [ARK Selections]

ALL THE BEST 😀:)